Summative Assessment 2:

Development Individual Project: Coding Output and Evidence of Testing

Secure Software Development

Word count: 1,390

GitHub repository:
https://github.com/MarqCodesFromZero/retail_store

Table of Contents

Abstract ***2***

Executive Summary ***2***

System Architecture ***3***

Overview **3**

Component Architecture **3**

Libraries and Frameworks Justification **4**

Security Implementation **5**

Authentication and Authorisation 5

Strategy Pattern: Secure vs Insecure Toggle 6

Password Security 7

Input Validation 7

Rate Limiting and Account Lockout 7

Encryption at Rest 8

GDPR Alignment 9

Adherence to Object-Oriented Coding Practices **10**
Encapsulation and Modularity 10
Inheritance and Polymorphism **10**
Dependency Injection 11
Testing Strategy ***12***
Future Recommendations ***14***
Conclusion ***14***
Reference List ***15***

Abstract

This paper describes the implementation of a secure retail store command-line application built in Python. The system demonstrates how security mechanisms can be toggled at runtime using the Strategy design pattern, allowing direct comparison between secure and insecure operation. The application implements authentication and TOTP-based multi-factor authentication, role-based access control, symmetric encryption for data at rest, rate limiting, and GDPR-aligned data anonymisation. It also includes testing across four test suites that validate both functionality and security resilience against brute-force, denial-of-service and injection attacks.

Executive Summary

The system is a Python CLI retail store that includes inventory management, user authentication, order processing, and account management while enforcing security controls that align with current digital identity and privacy-by-design guidance (Temoshok et al., 2025; Antunes and Guimaraes, 2024).

A key design decision is the switchable security strategy during runtime, that enables the ability to observe the consequences of removing protections without the need to restart the application.

Key features include:

- Bcrypt password hashing with TOTP multi-factor authentication
- Role-based access control with three distinct roles
- Input validation using regex whitelisting to prevent injection
- Fernet symmetric encryption (AES-128-CBC) for PII at rest
- Per-identity sliding-window rate limiting
- Account lockout after configurable failed login attempts
- Embedded Flask REST API with secure and insecure endpoint behaviour
- GDPR-aligned order anonymisation on account deletion

System Architecture

Overview

The application follows the Model-View-Controller (MVC) architectural pattern with a dedicated service layer. By separating concerns, the application ensures that business logic is isolated from presentation, improving maintainability and testability (Parnas,1972).

Component Architecture

The application has a modular design and separation of concerns:

```
app/
├── main.py            # Application entry point
├── config.py          # Application configuration and initialization
├── database.py        # ORM models and database schema
├── security.py        # Security configurations and utilities
├── managers.py        # Business logic and core operations
├── enums.py           # Enumerations for roles, statuses, etc.
├── exceptions.py      # Custom exception classes
├── api_routes.py      # Flask API route definitions
├── cli.py             # Command-line interface for admin and user actions
├── decorator.py       # Authentication and authorization decorators
└── tests/             # Test suite
```

Libraries and Frameworks Justification

```
bcrypt==4.0.1
cryptography==43.0.0
Flask==2.3.3
PyJWT==2.12.1
pyotp==2.9.0
requests==2.32.3
pytest==7.4.3
pycodestyle==2.11.1
pylint==4.0.5
```

The system uses six core dependencies:

- bcrypt provides adaptive password hashing resistant to brute-force attacks, aligning with OWASP password storage recommendations (Hatzivasilis, 2017).
- PyJWT enables stateless, signed session tokens using the HS256 algorithm.
- pyotp implements TOTP-based multi-factor authentication conforming to RFC 6238 (Temoshok et al., 2025).
- cryptography delivers authenticated encryption (AES-128-CBC with HMAC-SHA256) for customer PII at rest (NIST, 2023).
- Flask provides the REST API that serves as the target for attack simulations, chosen due to its lightweight design and wide ecosystem.

- requests handles external API calls to Frankfurter (currency conversion) and Stripe (payment processing).

Security Implementation

Authentication and Authorisation

The authentication flow applies defence in depth by combining password verification, TOTP-based MFA in secure mode, expiring JWT tokens, and a server-side active-session list that permits immediate revocation on logout. This combination is consistent with current NIST authentication guidance on multi-factor verification and session assurance (Temoshok et al., 2025).

The AuthService.require_role() method is the main authorisation gate, validating three conditions on every protected operation: the session token being in the active list, the associated account exists and has an active state, and the account's role is in the permitted set. This reflects zero-trust guidance that authentication and authorisation should be verified before access is granted, while also implementing a role-based access model (Rose et al., 2020; Sandhu, Ferraiolo and Kuhn, 2000).

```python
def require_role(self, token: str, allowed_roles: set[AccountRole]) -> Account:
    """Full auth check: active session + valid account + correct role.

    This is the primary authorisation gate used by controllers and
    the Flask API to protect operations.
    """
    if not self.datastore.has_session(token):
        raise AuthenticationError("Session is not active.")
    payload = self.decode_token(token)
    account = self.get_account(payload["sub"])
    self.policy.authorise(account, allowed_roles)
    if account is None:
        raise AuthenticationError("Account not found.")
    self.policy.ensure_account_active(account)
    return account
```

Strategy Pattern: Secure vs Insecure Toggle

The core architectural innovation is the Strategy pattern (Musch, 2023) applied to security controls. The abstract ISecurityPolicy interface defines seven security operations. SecurePolicy enforces all controls while on the contrary, InsecurePolicy deliberately bypasses them.

```python
class ISecurityPolicy(ABC):
    """Abstract interface that both secure and insecure policies implement."""

    @abstractmethod
    def validate_input(self, field_name: str, value: str) -> str:
        """Validate and sanitise user input; raise ValidationError on failure."""
        raise NotImplementedError

    @abstractmethod
    def authorise(self, account: Account | None, allowed_roles: set[AccountRole]) -> None:
        """Check that account holds one of the allowed RBAC roles."""
        raise NotImplementedError

    @abstractmethod
    def enforce_rate_limit(self, identity: str, limit: int, window_seconds: int) -> None:
        """Raise RateLimitError if identity exceeds limit within window."""
        raise NotImplementedError

    @abstractmethod
    def ensure_account_active(self, account: Account) -> None:
        """Block actions from locked or disabled accounts."""
        raise NotImplementedError

    @abstractmethod
    def validate_login_state(self, account: Account) -> None:
        """Check lockout state and auto-unlock expired lockouts."""
        raise NotImplementedError
```

This design avoids conditional checks spread across the codebase, keeping each module focused on a single responsibility (Parnas, 1972).

Password Security

```python
def validate_password(password: str) -> str:
    """Enforce NIST SP 800-63B password complexity requirements."""
    if not PASSWORD_PATTERN.fullmatch(password):
        raise ValidationError(
            "Password must be 8-64 chars and include upper, lower, digit, and special char."
        )
    return password
```

The implementation enforces passwords of 8-64 characters and requires uppercase, lowercase, digit, and special-character classes. The minimum-length baseline and verifier-side resistance to online guessing are consistent with current NIST guidance, while bcrypt provides an adaptive password-hashing mechanism for stored credentials (Temoshok et al., 2025; Hatzivasilis, 2017).

Input Validation

```python
def validate_safe_text(value: str, field_name: str = "value") -> str:
    """Reject input containing SQL/XSS injection metacharacters."""
    if not SAFE_TEXT_PATTERN.fullmatch(value):
        raise ValidationError(f"Invalid characters detected in {field_name}.")
    return value
```

All user input in secure mode passes through regex-based validators before reaching business logic. The validate_safe_text() function uses a whitelist pattern that blocks common SQL and XSS characters. This aligns with OWASP's recommendation that input validation is one of the most effective defences against injection attacks (OWASP , 2025)

Rate Limiting and Account Lockout

```python
def enforce_rate_limit(self, identity: str, limit: int, window_seconds: int) -> None:
    """Sliding-window rate limiter that blocks after limit hits per window."""
    now = time()
    window = [ts for ts in self._requests[identity] if now - ts < window_seconds]
    window.append(now)
    self._requests[identity] = window
    if len(window) > limit:
        self._logger.warning("Rate limit triggered for identity=%s", identity)
        raise RateLimitError("Rate limit exceeded.")
```

The SecurePolicy.enforce_rate_limit() method implements a sliding-window algorithm that tracks request timestamps per identity. Login attempts are capped at five per minute; exceeding the threshold raises RateLimitError. After three consecutive failed logins, the account transitions to LOCKED status with a configured five-minute lockout window. Auto-unlock occurs on the next login attempt after the window expires, maintaining usability (Temoshok et al., 2025).

Encryption at Rest

Customer addresses are encrypted using Fernet (AES-128-CBC with HMAC-SHA256 authentication) via the EncryptionService. The encryption key must be loaded from the FERNET_KEY environment variable, otherwise a missing key causes a fail-fast startup error rather than silently storing plaintext. In insecure mode, the AuthService bypasses encryption entirely, making the contrast visible in the JSON datastore (NIST, 2023).

```python
class EncryptionService:
    """Provides encrypt/decrypt operations using Fernet symmetric encryption."""

    def __init__(self, key: str | None = None, env_var: str = "FERNET_KEY") -> None:
        """Initialise with an explicit key or from an environment variable.

        Raises ValueError if no key is available — fail-fast prevents
        accidental plaintext storage in production.
        """
        resolved = key or os.getenv(env_var)
        if not resolved:
            raise ValueError(
                f"Missing required secret: set the {env_var} environment variable "
                "(see .env.example for setup guidance)."
            )
        self._fernet = Fernet(resolved.encode("utf-8"))

    def encrypt(self, plaintext: str) -> str:
        """Encrypt a string and return the Fernet token as UTF-8 text."""
        if not plaintext:
            return plaintext
        return self._fernet.encrypt(plaintext.encode("utf-8")).decode("utf-8")

    def decrypt(self, ciphertext: str) -> str:
        """Decrypt a Fernet token back to the original plaintext string."""
        if not ciphertext:
            return ciphertext
        return self._fernet.decrypt(ciphertext.encode("utf-8")).decode("utf-8")
```

GDPR Alignment

When a customer account is deleted, the system anonymises all associated orders by replacing the customer email with a “deleted”, therefore preserving order information. Audit logs also use a pseudonymised email format (a***@domain.com) to minimise PII exposure. No passwords, OTPs, or JWT tokens are written to log files, aligning with GDPR Article 5(1)(c) data minimisation principles. These choices reflect privacy-by-design practice and reduce linkability of stored personal data (Antunes and Guimaraes, 2024; Neumann et al., 2019).

```
2026-04-08 16:04:50,785 INFO AuthService Staff registered email=a***@retail.local role=ADMINISTRATOR
```

Adherence to Object-Oriented Coding Practices

Encapsulation and Modularity

Certain classes such as services encapsulate functionality, isolating business logic from implementation. This reflects the modular design principle of separating volatile concerns behind stable interfaces (Parnas, 1972).

```python
class CatalogService:
    """Manages product lifecycle: creation, update, search, deletion, and stock."""

    def __init__(self, datastore: DataStore) -> None:
        self.datastore = datastore

    def _next_product_id(self) -> str:
        """Generate the next sequential PRD### identifier."""
        existing_numbers: list[int] = []
        for item_id in self.datastore.list_products():
            if item_id.startswith("PRD") and item_id[3:].isdigit():
                existing_numbers.append(int(item_id[3:]))
        next_number = max(existing_numbers, default=0) + 1
        return f"PRD{next_number:03d}"

    def add_product(self, product: Product) -> None:
        """Persist a product to the datastore."""
        self.datastore.upsert_product(product.item_id, product_to_dict(product))
```

Inheritance and Polymorphism

The account hierarchy uses an abstract base class Account with different subclasses Customer, Clerk, and Administrator, enabling polymorphic authentication while allowing role-specific controllers to narrow types via isinstance checks. The exception hierarchy follows the same pattern: RetailSystemError as the base, with ValidationError, AuthenticationError, AuthorizationError, RateLimitError, and ExternalServiceError as specialisations.

```python
@dataclass(slots=True)
class Account(ABC):
    """Base account shared by every role (Customer, Clerk, Administrator)."""
    email: str                                  # Unique identifier / login name
    password_hash: str                          # bcrypt hash (secure) or plaintext (insecure)
    role: AccountRole                           # RBAC role for authorisation checks
    status: AccountStatus = AccountStatus.ACTIVE  # Controls login eligibility
    mfa_secret: str = ""                        # TOTP shared secret (RFC 6238)
    failed_attempts: int = 0                    # Counter for brute-force lockout
    locked_until: str = ""                      # ISO-8601 timestamp; empty = not locked
```

```python
@dataclass(slots=True)
class Customer(Account):
    """Customer account. Can browse, order and manage their address."""
    saved_address: str = ""                     # Fernet-encrypted in secure mode
    order_history: list[str] = field(default_factory=list)  # List of order IDs
```

```python
@dataclass(slots=True)
class Clerk(Account):
    """Clerk account. Can manage the product catalogue."""
    employee_id: str = ""                       # Auto-generated EMP### identifier
```

```python
@dataclass(slots=True)
class Administrator(Account):
    """Administrator account. Has full system access including account management."""
    admin_level: int = 1                        # Higher = broader permissions
```

Dependency Injection

All service constructors accept collaborators as parameters rather than instantiating them internally. The build_controller() factory in app.py wires the dependency at startup, allowing for isolated unit testing, as each test can inject mock or test-specific instances.

```python
def build_controller() -> AppController:
    """Wire up all services and return a fully configured AppController."""
    configure_logging()
    datastore = DataStore()
    encryption_service = EncryptionService()
    secure_policy = SecurePolicy()
    insecure_policy = InsecurePolicy()
    auth_service = AuthService(
        datastore=datastore,
        encryption_service=encryption_service,
        active_policy=secure_policy,
        secure_policy=secure_policy,
        insecure_policy=insecure_policy,
    )
    catalog_service = CatalogService(datastore=datastore)
    currency_service = CurrencyService()
    payment_service = PaymentService()
    order_service = OrderService(
        datastore=datastore,
        catalog_service=catalog_service,
        currency_service=currency_service,
        payment_service=payment_service,
    )
    return AppController(
        auth_service=auth_service,
        catalog_service=catalog_service,
        order_service=order_service,
        datastore=datastore,
    )
```

Testing Strategy

The testing strategy follows a segmented approach, with four types of tests totalling 76 test cases. Static code analysis is made using pycodestyle and pylint, which verify PEP-8 compliance and code quality, which Bennett et al. (2024) highlight as essential for maintaining consistent standards.

The test suites are structured as follows:

Suite	Tests	Coverage
test_unit.py	34	Password hashing, input validation, rate limiting, policy toggle, menu routing, CRUD, deletion

Suite	Tests	Coverage
test_integration.py	5	DataStore persistence, Frankfurter API, Stripe API, payment simulation fallback
test_acceptance.py	15	Full customer checkout, brute-force lockout, CRUD flows, deletion workflows
test_system.py	22	End-to-end secure vs insecure: brute force, DoS, injection, session invalidation

All tests use deterministic secrets and isolated temporary datastores, requiring no environment configuration. This approach make sure that there is existing validation of functionality, alongside security resilience, consistent with the shift-left testing methodology advocated by Bozic and Wotawa (2020). The system tests specifically exercise the three attack simulations (brute force, DoS, injection) in both security modes, confirming that secure mode blocks and insecure mode permits each attack vector.

```
● (base) rubenmarques@unknown6293cf7e036f retail_store % python -m pytest tests/ -q
  ..................................................................... [ 85%]
  ...........                                                           [100%]
  ============================================================== warnings summary ==============================================================
  tests/test_unit.py: 22 warnings
    /opt/anaconda3/lib/python3.12/site-packages/jwt/api_jwt.py:147: InsecureKeyLengthWarning: The HMAC key is 30 bytes long, which is below the minimum recommended length of 32 bytes for SHA2
  56. See RFC 7518 Section 3.2.
      return self._jws.encode(

  tests/test_unit.py: 16 warnings
    /opt/anaconda3/lib/python3.12/site-packages/jwt/api_jwt.py:365: InsecureKeyLengthWarning: The HMAC key is 30 bytes long, which is below the minimum recommended length of 32 bytes for SHA2
  56. See RFC 7518 Section 3.2.
      decoded = self.decode_complete(

  -- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
  76 passed, 38 warnings in 31.48s
(base) rubenmarques@unknown6293cf7e036f retail_store %
```

```
● (base) rubenmarques@unknown6293cf7e036f retail_store % python -m pylint --max-line-length=150 app.py controllers/*.py models/*.py services/*.py utils/*.py views/*.py data/datastore.py

  ------------------------------------------------------------------
  Your code has been rated at 10.00/10 (previous run: 0.00/10, +10.00)
```

```
● (base) rubenmarques@Mac retail_store % conda activate base
● (base) rubenmarques@Mac retail_store % python -m pycodestyle --max-line-length=120 app.py controllers/ models/ services/ utils/ views/ data/datastore.py
○ (base) rubenmarques@Mac retail_store %
```

Future Recommendations

- Implement password reset functionality: Introducing email-verified password reset endpoints would improve usability without compromising the authentication model.
- Migrate to a relational database: Replacing JSON file persistence with SQLAlchemy and SQLite would provide proper ACID transactions, concurrent access, and prepared statements for production-grade injection prevention.
- Add automated security scanning: Integrating tools such as Bandit into the CI/CD pipeline would enable continuous detection of security-sensitive code patterns (Bennett et al., 2024).
- Enhance logging and monitoring: Adopting structured JSON logging with centralised aggregation would support real-time intrusion detection and incident response.

Conclusion

The retail store CLI demonstrates a contemporary approach to building secure Python applications, integrating authentication, access control, encryption, and audit logging as first-class concerns rather than afterthoughts. The Strategy pattern enables direct comparison of secure and insecure behaviour, making the impact of each security control observable and measurable. Comprehensive testing across four layers validates both functional correctness and security resilience, while adherence to object-oriented principles ensures the codebase remains maintainable and extensible.

Reference List

Antunes, P. and Guimaraes, N. (2024) ‘Guiding the implementation of data privacy with microservices’, *International Journal of Information Security*, 23, pp. 3591-3608. doi:10.1007/s10207-024-00907-y

Bennett, G., Hall, T., Counsell, S., Winter, E. and Shippey, T. (2024) ‘Do Developers Use Static Application Security Testing (SAST) Tools Straight Out of the Box? A large-scale Empirical Study’, in *Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*, pp. 454-460. doi:10.1145/3674805.3690750

Bozic, J. and Wotawa, F. (2020) ‘Planning-based security testing of web applications with attack grammars’, *Software Quality Journal*, 28(1), pp. 307-334. doi:10.1007/s11219-019-09469-y

Hatzivasilis, G. (2017) ‘Password-Hashing Status’, *Cryptography*, 1(2), 10. doi:10.3390/cryptography1020010

Jones, M., Bradley, J. and Sakimura, N. (2015) *JSON Web Token (JWT)*. RFC 7519. doi:10.17487/RFC7519

M'Raihi, D., Machani, S., Pei, M. and Rydell, J. (2011) *TOTP: Time-Based One-Time Password Algorithm*. RFC 6238. doi:10.17487/RFC6238

Musch, O. (2023) *Design Patterns with Java: An Introduction*. Wiesbaden: Springer Vieweg. doi:10.1007/978-3-658-39829-3

Neumann, G.K., Grace, P., Burns, D. and Surridge, M. (2019) ‘Pseudonymization risk analysis in distributed systems’, *Journal of Internet Services and Applications*, 10, 1. doi:10.1186/s13174-018-0098-z

NIST (2023) *Advanced Encryption Standard (AES)*. FIPS 197-upd1. doi:10.6028/NIST.FIPS.197-upd1

NIST (2025) *Digital Identity Guidelines: Authentication and Authenticator Management*. SP 800-63B-4. doi:10.6028/NIST.SP.800-63B-4

OWASP (2025a) *Authentication Cheat Sheet*. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

OWASP (2025b) *Input Validation Cheat Sheet*. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html

OWASP (2025c) *Password Storage Cheat Sheet*. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

OWASP (2025d) *Cryptographic Storage Cheat Sheet*. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html

Parnas, D.L. (1972) 'On the criteria to be used in decomposing systems into modules', *Communications of the ACM*, 15(12), pp. 1053-1058. doi:10.1145/361598.361623

Rose, S., Borchert, O., Mitchell, S. and Connelly, S. (2020) *Zero Trust Architecture*. NIST SP 800-207. doi:10.6028/NIST.SP.800-207

Sandhu, R., Ferraiolo, D. and Kuhn, D. (2000) 'The NIST Model for Role-Based Access Control: Towards a Unified Standard', in *Proceedings of the Fifth ACM Workshop on Role-Based Access Control*, pp. 47-63. doi:10.1145/344287.344301